

ESCUELA DE INGENIERÍA INFORMÁTICA DE OVIEDO

Arquitectura de Computadores

Curso 2025-2026

Teamwork

Díaz Mendaña, Diego - UO301887

García Pernas, Pablo - UO300167

Rama García, Mateo - UO300710

Suárez Fernández, Fernando - UO300028

Grupo de prácticas: PL.1 - B

Titulación: PCEO Informática y Matemáticas

7 de octubre de 2025

Índice

1. Introducción	2
2. Desarrollo del algoritmo	2
2.1. Comprobaciones e inicialización	2
2.2. Implementación del algoritmo	4
3. Desarrollo del Proyecto	5
3.1. Fase 1: Implementación Monohilo	5
4. Resultados y Análisis	5
5. Conclusiones	5
A. Flujo de trabajo y metodología de desarrollo	6
A.1. Metodología	6

1. Introducción

La presente memoria recoge el desarrollo del proyecto correspondiente a la asignatura *Arquitectura de Computadores*, impartida durante el curso académico 2025-2026 en la Escuela de Ingeniería Informática de Oviedo. El objetivo del proyecto es aplicar los conocimientos adquiridos en la asignatura para diseñar e implementar un filtro de imágenes en lenguaje C/C++. Asimismo, se realiza un análisis del rendimiento del filtro y una comparación entre las versiones monohilo y multihilo.

El trabajo se ha estructurado en distintas fases progresivas:

- **Entrega 1 (Monohilo):** Implementación básica del filtro de imágenes en un único hilo de ejecución.
- **Entrega 2 (SIMD y Multihilo):** Optimización del filtro mediante el uso de instrucciones SIMD y la paralelización con múltiples hilos.

2. Desarrollo del algoritmo

Se desarrolló el algoritmo de inversión en blanco y negro (Algoritmo número #3). El desarrollo se realizó utilizando la máquina virtual proporcionada por el profesorado, junto con la plantilla base facilitada.

La plantilla incluía las líneas de código necesarias para obtener el número de píxeles, así como los punteros a los arreglos RGB de las imágenes de entrada y salida. Por tanto, fue necesario únicamente implementar las comprobaciones y el algoritmo principal.

2.1. Comprobaciones e inicialización

Antes de implementar el algoritmo, se llevaron a cabo las comprobaciones necesarias para verificar la existencia del archivo de imagen y la correcta inicialización de las variables. La comprobación de existencia se realiza mediante la siguiente función:

```
1 CImg<data_t> load_image(const char* filename) {  
2     if (access(filename, F_OK) == -1) {  
3         fprintf(stderr, "Error: la imagen '%s' no existe.\n", filename);  
4         exit(EXIT_FAILURE);  
5     }  
6  
7     CImg<data_t> image(filename);  
8     return image;  
9 }
```

Esta función recibe como parámetro el nombre del archivo y emplea la llamada `access()` con el parámetro `F_OK`. Si la imagen no existe, la función devuelve `-1`, se muestra un mensaje de error en la salida estándar y finaliza la ejecución del programa.

En caso de existir el archivo, se crea un objeto de la clase `CImg` y se devuelve. Adicionalmente, se comprueba que las dimensiones y el espectro de la imagen de salida coincidan con los de la imagen original:

```
1  if (static_cast<unsigned int>(dstImage.width()) != width ||
2      static_cast<unsigned int>(dstImage.height()) != height ||
3      static_cast<unsigned int>(dstImage.spectrum()) != nComp) {
4      fprintf(stderr, "Error: las dimensiones de la imagen de salida no coinciden con la
        original.\n");
5      free(pDstImage);
6      exit(EXIT_FAILURE);
7  }
```

En caso de discrepancia, se muestra un mensaje de error y se detiene la ejecución del programa.

Asimismo, se modificó el tipo de dato empleado de `float` a `double`. Se definieron también los pesos para las componentes de color y el brillo máximo, como se muestra a continuación:

```
1  #define R_WEIGHT 0.3
2  #define G_WEIGHT 0.59
3  #define B_WEIGHT 0.11
4  #define MAX_BRIGHTNESS 255.0
```

Para medir el tiempo de ejecución, se emplearon las funciones `clock_gettime()` con los parámetros `CLOCK_REALTIME` y las direcciones de memoria de las variables temporales. En caso de error, se muestra un mensaje y se finaliza la ejecución:

```
1  if(clock_gettime(CLOCK_REALTIME, &tStart) == -1) {
2      printf("Error al obtener el tiempo inicial");
3      exit(EXIT_FAILURE);
4  }
5
6  if(clock_gettime(CLOCK_REALTIME, &tEnd) == -1){
7      printf("Error al obtener el tiempo final");
8      exit(EXIT_FAILURE);
9  }
```

Finalmente, se calcula el tiempo transcurrido y se muestra por la salida estándar:

```
1  elapsedTime = (tEnd.tv_sec - tStart.tv_sec) +
2                (tEnd.tv_nsec - tStart.tv_nsec) / 1e9;
3  printf("Elapsed time (%d repetitions): %.6f seconds\n",
4        N_ITERATIONS, elapsedTime);
```

2.2. Implementación del algoritmo

El algoritmo implementado corresponde al filtro de inversión en escala de grises. Para cada píxel de la imagen, la función `filter` calcula una luminosidad ponderada a partir de las componentes RGB, utilizando los coeficientes definidos previamente. El valor resultante se invierte respecto al brillo máximo y se asigna a las tres componentes del píxel en la imagen de salida:

```
1 void filter (filter_args_t args) {
2     for (uint i = 0; i < args.pixelCount; i++) {
3         data_t L = *(args.pRsrc + i) * R_WEIGHT +
4                     *(args.pGsrc + i) * G_WEIGHT +
5                     *(args.pBsrc + i) * B_WEIGHT;
6
7         L = MAX_BRIGHTNESS - L;
8
9         *(args.pRdst + i) = L;
10        *(args.pGdst + i) = L;
11        *(args.pBdst + i) = L;
12    }
13 }
```

Dado que el tiempo de ejecución debía situarse entre 8 y 15 segundos, se realizó un bucle de 125 iteraciones de la función `filter` para mantener el tiempo dentro del rango esperado:

```
1 for (int i = 0; i < N_ITERATIONS; i++) {
2     filter(filter_args);
3 }
```

A continuación, se muestran la imagen original y la imagen resultante tras aplicar el filtro:



Figura 1: Imagen original.



Figura 2: Imagen resultante (versión monohilo).

3. Desarrollo del Proyecto

3.1. Fase 1: Implementación Monohilo

En la primera fase del proyecto se desarrolló la versión monohilo del algoritmo, centrada en la comprensión del flujo de procesamiento y en la verificación de la corrección del filtro aplicado sobre las imágenes.

4. Resultados y Análisis

(Se deben incluir los tiempos de ejecución, comparativas entre versiones y gráficos de rendimiento.)

5. Conclusiones

(Se deben exponer los principales aprendizajes, la mejora de rendimiento observada y las implicaciones del uso de SIMD y multihilo en procesamiento de imágenes.)

ANEXOS

A. Flujo de trabajo y metodología de desarrollo

Todo el desarrollo del proyecto se encuentra disponible en el repositorio de *GitHub*: [ver repositorio](#). Para garantizar un trabajo colaborativo eficiente, se estableció un flujo de trabajo común basado en las herramientas *Git* y *GitHub* como elementos centrales de control de versiones. Los aspectos clave son:

- **Repositorio compartido:** Contiene el código, la documentación y los recursos del proyecto.
- **Issues:** Cada tarea se define como una *issue* en *GitHub*, asignada a un responsable con una fecha límite.
- **Project Board:** Se emplea un tablero *Kanban* con las columnas *To Do*, *In Progress*, *Review* y *Done* para visualizar el progreso del trabajo.
- **Milestones:** Cada entrega del proyecto se asocia a un *milestone* que agrupa las *issues* relacionadas.
- **Pull Requests:** Todas las contribuciones se integran mediante *pull requests*, revisadas y aprobadas por, al menos, tres de los cuatro miembros del equipo antes de fusionarse en la rama principal.
- **Documentación:** Se conserva en la carpeta `docs/`.

A.1. Metodología

El equipo adoptó una metodología ágil adaptada al contexto académico, inspirada en un enfoque híbrido entre *Scrum* y *Kanban*. Los elementos principales de esta metodología son:

- La planificación se organiza en **iteraciones** (*milestones*) que corresponden a las entregas del proyecto.
- Las tareas se dividen en **issues** manejables, priorizadas según su relevancia y dependencias, y se asignan a los miembros del equipo.
- El **Project Board** facilita la visualización del progreso y la gestión eficiente de las tareas.
- La revisión colectiva de los *pull requests* promueve la calidad del código y el aprendizaje conjunto.
- Las dependencias entre tareas se gestionan mediante etiquetas y referencias cruzadas en *GitHub*.